Lab 03

Lab 03: Spatial autocorrelation, globally and locally

Read the instructions COMPLETELY before starting the lab

This lab builds on many of the discussions and exercises from class, including previous labs.

Attribution

This lab uses some code examples and directions from https://mgimond.github.io/Spatial/spatial-autocorrelation-in-r.html

Formatting your submission

This lab must be placed into a public repository on GitHub (www.github.com). Before the due date, submit **on Canvas** a link to the repository. I will then download your repositories and run your code. The code must be contained in either a .R script or a .Rmd markdown document. As I need to run your code, any data you use in the lab must be referenced using **relative path names**. Finally, answers to questions I pose in this document must also be in the repository at the time you submit your link to Canvas. They can be in a separate text file, or if you decide to use an RMarkdown document, you can answer them directly in the doc.

Data

The data for this lab can be found on the US Census website. The US Census provides various files and tables for download, but will instead use a package that provides a convenient way to access Census data and geometry. This will REQUIRE YOU TO REGISTER FOR AN API KEY and then use it in your code. You can read more about tidycensus and how to register and use your API key here: https://walker-data.com/tidycensus/articles/basic-usage.html

NOTE, you can use your API key using the following command `census_api_key("YOUR API KEY GOES HERE")`. BUT YOU SHOULD **NOT** INCLUDE THAT COMMAND IN YOUR R SCRIPT. I want to be VERY clear - if you put your API in the R script and then publish the code (e.g., to GitHub), someone else can use your API key. You do NOT want that. Instead, use the command in the console, and `tidycensus` will write the API key to your R Environmental variables file.

Introduction

In this lab, we will be calculating the spatial autocorrelation of various Census variables across a subset of the US. Will be using the `tidycensus` pacakge to download data, and we will also be using a new package called `spdep` in this assignment.

We begin by loading the relevant packages and data

```r
library(spdep)
```

```
## Warning: package 'spdep' was built under R version 4.4.3

## Loading required package: spData

## Warning: package 'spData' was built under R version 4.4.1

## Loading required package: sf

## Warning: package 'sf' was built under R version 4.4.3

## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

```r
library(tidycensus)
```

```
## Warning: package 'tidycensus' was built under R version 4.4.3
```

```r
library(sf)
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3

## Warning: package 'tibble' was built under R version 4.4.3

## Warning: package 'tidyr' was built under R version 4.4.3

## Warning: package 'readr' was built under R version 4.4.3

## Warning: package 'purrr' was built under R version 4.4.3

## Warning: package 'dplyr' was built under R version 4.4.3

## Warning: package 'stringr' was built under R version 4.4.1

## Warning: package 'forcats' was built under R version 4.4.1

## Warning: package 'lubridate' was built under R version 4.4.3

## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tmap)
```

```
## Warning: package 'tmap' was built under R version 4.4.1
```

The tidycenus packages uses an Application Programming Interface, or API, to download data directly from the US Census and then process into an R-friendly data structure.

First, let's use tidycensus to download data for all US states. Review the `get_acs` documentation to understand what parameters are available to you and what they do.

```r
options(tigris_use_cache = TRUE) # cache geometry so you don't dl unnecessarily

d.all <- get_acs(geography = "county",
                      table = "B15003",
                      output = "wide",
                      geometry = T,
                      year = 2020)
```

```
## Getting data from the 2016-2020 5-year ACS
```

```r
glimpse(d.all)
```

```
## Rows: 3,221
## Columns: 53
## $ GEOID       <chr> "13031", "13121", "13179", "13189", "13213", "13209", "131~
## $ NAME        <chr> "Bulloch County, Georgia", "Fulton County, Georgia", "Libe~
## $ B15003_001E <dbl> 43788, 716013, 36054, 14403, 26609, 6029, 132146, 132953, ~
## $ B15003_001M <dbl> 107, 112, 85, 248, 106, 84, 134, 153, 311, 234, 266, 134, ~
## $ B15003_002E <dbl> 270, 6548, 327, 133, 845, 76, 5041, 1437, 190, 223, 1160, ~
## $ B15003_002M <dbl> 112, 1017, 131, 65, 212, 55, 684, 337, 108, 109, 364, 44, ~
## $ B15003_003E <dbl> 1, 240, 50, 0, 0, 0, 31, 11, 11, 0, 24, 0, 0, 0, 0, 0, 0, ~
## $ B15003_003M <dbl> 3, 290, 74, 26, 30, 20, 46, 23, 15, 20, 34, 20, 22, 20, 26~
## $ B15003_004E <dbl> 0, 360, 0, 0, 17, 1, 11, 90, 40, 0, 0, 0, 0, 0, 0, 0, 0, 2~
## $ B15003_004M <dbl> 33, 180, 33, 26, 31, 2, 18, 81, 59, 20, 33, 20, 22, 20, 26~
## $ B15003_005E <dbl> 0, 170, 0, 44, 23, 0, 140, 26, 0, 0, 0, 5, 0, 0, 1, 0, 0, ~
## $ B15003_005M <dbl> 33, 140, 33, 74, 35, 20, 120, 33, 22, 20, 33, 9, 22, 20, 3~
## $ B15003_006E <dbl> 0, 289, 9, 0, 23, 0, 303, 3, 0, 6, 40, 0, 0, 14, 0, 0, 0, ~
## $ B15003_006M <dbl> 33, 180, 15, 26, 22, 20, 218, 5, 22, 12, 54, 20, 22, 23, 2~
## $ B15003_007E <dbl> 7, 656, 0, 13, 179, 19, 599, 297, 12, 0, 92, 15, 28, 24, 0~
## $ B15003_007M <dbl> 11, 324, 33, 22, 127, 26, 200, 206, 16, 20, 82, 19, 48, 27~
## $ B15003_008E <dbl> 66, 446, 64, 14, 91, 21, 788, 100, 18, 80, 41, 16, 38, 27,~
## $ B15003_008M <dbl> 53, 186, 61, 23, 73, 25, 298, 89, 27, 60, 48, 21, 59, 33, ~
## $ B15003_009E <dbl> 51, 1140, 22, 0, 156, 0, 974, 218, 15, 83, 51, 13, 80, 127~
## $ B15003_009M <dbl> 56, 270, 28, 26, 83, 20, 358, 119, 20, 60, 45, 19, 46, 70,~
## $ B15003_010E <dbl> 167, 2485, 197, 120, 723, 9, 4682, 540, 77, 77, 317, 45, 7~
## $ B15003_010M <dbl> 121, 546, 106, 140, 248, 17, 695, 180, 48, 63, 155, 31, 37~
## $ B15003_011E <dbl> 213, 1508, 50, 63, 236, 6, 966, 618, 157, 218, 259, 5, 65,~
## $ B15003_011M <dbl> 105, 358, 43, 50, 117, 11, 281, 192, 123, 123, 98, 9, 41, ~
## $ B15003_012E <dbl> 285, 2954, 181, 189, 606, 144, 1610, 1518, 342, 126, 466, ~
## $ B15003_012M <dbl> 126, 500, 87, 110, 185, 62, 311, 378, 116, 77, 135, 33, 94~
## $ B15003_013E <dbl> 835, 5062, 493, 449, 1157, 122, 2894, 2221, 171, 228, 1037~
## $ B15003_013M <dbl> 271, 707, 139, 180, 254, 55, 535, 383, 80, 93, 293, 57, 14~
```

```
## $ B15003_014E <dbl> 789, 7551, 552, 402, 1211, 345, 3296, 4140, 340, 354, 1401~
## $ B15003_014M <dbl> 235, 752, 130, 163, 270, 104, 430, 606, 155, 116, 303, 88,~
## $ B15003_015E <dbl> 1779, 11023, 526, 431, 1161, 219, 2953, 4631, 317, 532, 72~
## $ B15003_015M <dbl> 425, 1285, 158, 152, 309, 69, 454, 722, 131, 202, 175, 115~
## $ B15003_016E <dbl> 715, 9168, 467, 386, 951, 43, 3205, 3121, 153, 104, 1374, ~
## $ B15003_016M <dbl> 236, 1011, 132, 188, 250, 24, 494, 489, 80, 57, 307, 36, 1~
## $ B15003_017E <dbl> 10493, 108552, 8910, 4491, 6857, 1947, 30686, 34840, 2755,~
## $ B15003_017M <dbl> 868, 4220, 607, 528, 573, 198, 1301, 1560, 373, 302, 929, ~
## $ B15003_018E <dbl> 2035, 15352, 1752, 1336, 2248, 350, 6392, 7800, 1156, 750,~
## $ B15003_018M <dbl> 339, 1259, 282, 298, 333, 108, 779, 819, 336, 275, 524, 14~
## $ B15003_019E <dbl> 2757, 25672, 3240, 1134, 1772, 369, 8451, 7373, 491, 167, ~
## $ B15003_019M <dbl> 420, 1631, 489, 296, 371, 91, 691, 755, 159, 84, 476, 77, ~
## $ B15003_020E <dbl> 7440, 83249, 7994, 1853, 3586, 785, 18530, 22324, 1248, 98~
## $ B15003_020M <dbl> 847, 3093, 647, 325, 518, 157, 1398, 1469, 230, 234, 659, ~
## $ B15003_021E <dbl> 3793, 43295, 4266, 1182, 1943, 580, 8898, 11742, 632, 455,~
## $ B15003_021M <dbl> 586, 2051, 412, 273, 417, 142, 686, 1036, 180, 143, 500, 1~
## $ B15003_022E <dbl> 6865, 229596, 4767, 1255, 1967, 546, 19459, 18388, 654, 47~
## $ B15003_022M <dbl> 702, 4598, 541, 257, 358, 117, 1050, 1378, 257, 150, 712, ~
## $ B15003_023E <dbl> 3276, 115136, 1747, 680, 621, 334, 7802, 7028, 178, 115, 2~
## $ B15003_023M <dbl> 601, 2933, 329, 217, 194, 125, 746, 904, 86, 66, 352, 77, ~
## $ B15003_024E <dbl> 747, 30592, 266, 101, 171, 91, 2713, 2782, 222, 21, 779, 1~
## $ B15003_024M <dbl> 263, 1965, 108, 67, 78, 76, 416, 421, 148, 26, 258, 61, 85~
## $ B15003_025E <dbl> 1204, 14969, 174, 127, 65, 22, 1722, 1705, 140, 33, 289, 3~
## $ B15003_025M <dbl> 320, 1263, 66, 99, 49, 20, 371, 425, 108, 33, 88, 35, 114,~
## $ geometry    <MULTIPOLYGON [°]> MULTIPOLYGON (((-82.02684 3..., MULTIPOLYGON ~
```

```r
#tmap::tm_shape(d.all) + tm_polygons() # commented out because
#it's a large dataset that takes a long time to plot, but you should try it
```

The above code, we downloaded the `B15003` table. Where you see the a variable noted with an `E`, that's the estimate. An `M` indicates the Margin of Error for that estimate.

But a tricky thing about using `tidycensus` is that you need to know which table and variables you want to download ahead of time, and there are a LOT of data in the Census database. Conveniently, there is a way to download the variables so that we can download just what we want.

```r
df.vars <- load_variables(2020, "acs5", cache = T)

glimpse(df.vars)
```

```
## Rows: 27,850
## Columns: 4
## $ name      <chr> "B01001A_001", "B01001A_002", "B01001A_003", "B01001A_004", ~
## $ label     <chr> "Estimate!!Total:", "Estimate!!Total:!!Male:", "Estimate!!To~
## $ concept   <chr> "SEX BY AGE (WHITE ALONE)", "SEX BY AGE (WHITE ALONE)", "SEX~
## $ geography <chr> "tract", "tract", "tract", "tract", "tract", "tract", "tract~
```

You can review, search, and filter those data to find any number of variables. But for the purpose of this lab, let's find the variable the corresponds to the estimate for the female population that is 75-79 years old. There are many ways we can do this, so let's explore

```r
df.vars %>% dplyr::filter(concept == "SEX BY AGE") %>%
  # If you break the chain here, you'll see a group of vars
  dplyr::filter(str_detect(label, "Female")) %>% # narrow to Female estimates
  dplyr::filter(str_detect(label, "75")) ## then to the variable we want
```

```
## # A tibble: 1 x 4
##   name       label                                      concept    geography
##   <chr>      <chr>                                      <chr>      <chr>
## 1 B01001_047 Estimate!!Total:!!Female:!!75 to 79 years SEX BY AGE block group
```

Now that we know the variable of interest, we can either download JUST that variable, or we can download the whole table. In the above example, the table is `B01001` and the variable number is `047`. It's usually easier to download the whole table in case you want other variables as well (and you often do), but either way works.

```r
d.sex.by.age <- get_acs(geography = "county",
                        table = "B01001",
                        output = "wide",
                        geometry = T,
                        year = 2020)
```

```
## Getting data from the 2016-2020 5-year ACS
```

```r
glimpse(d.sex.by.age)
```

```
## Rows: 3,221
## Columns: 101
## $ GEOID       <chr> "13031", "13121", "13179", "13189", "13213", "13209", "131~
## $ NAME        <chr> "Bulloch County, Georgia", "Fulton County, Georgia", "Libe~
## $ B01001_001E <dbl> 77719, 1051550, 62039, 21404, 39789, 9069, 201434, 202178,~
## $ B01001_001M <dbl> NA, NA, NA, NA, NA, NA, 21, NA, NA, NA, NA, NA, NA, NA, NA~
## $ B01001_002E <dbl> 37941, 508988, 31316, 9883, 19563, 4554, 100169, 97848, 76~
## $ B01001_002M <dbl> 196, 90, 86, 242, 69, 92, 99, 280, 125, 261, 165, 88, 164,~
## $ B01001_003E <dbl> 2364, 31142, 3265, 547, 1249, 227, 6647, 7127, 352, 131, 2~
## $ B01001_003M <dbl> 116, 58, 61, 197, 13, 38, 80, 198, 82, 129, 128, 25, 23, 5~
## $ B01001_004E <dbl> 1677, 32815, 2019, 778, 1363, 206, 6965, 6893, 216, 130, 2~
## $ B01001_004M <dbl> 238, 1252, 261, 223, 231, 51, 469, 690, 83, 83, 300, 57, 1~
## $ B01001_005E <dbl> 2469, 32425, 2488, 711, 1441, 357, 7876, 5836, 486, 160, 2~
## $ B01001_005M <dbl> 229, 1245, 287, 224, 228, 79, 463, 697, 83, 95, 304, 55, 1~
## $ B01001_006E <dbl> 1239, 19745, 1012, 459, 909, 98, 4678, 3770, 134, 82, 1518~
## $ B01001_006M <dbl> 30, 49, 54, 107, 19, 44, 51, 132, 52, 84, 76, 39, 53, 25, ~
## $ B01001_007E <dbl> 3404, 15818, 986, 137, 486, 339, 2524, 3630, 480, 43, 830,~
## $ B01001_007M <dbl> 222, 64, 103, 122, 57, 79, 136, 157, 300, 31, 149, 89, 21,~
## $ B01001_008E <dbl> 1862, 7192, 760, 256, 291, 85, 1399, 1955, 65, 36, 260, 72~
## $ B01001_008M <dbl> 357, 719, 205, 156, 136, 48, 292, 443, 100, 25, 111, 54, 5~
## $ B01001_009E <dbl> 1503, 7315, 950, 110, 177, 83, 1212, 1998, 104, 56, 400, 5~
## $ B01001_009M <dbl> 303, 933, 248, 89, 96, 48, 255, 497, 75, 45, 142, 45, 87, ~
## $ B01001_010E <dbl> 2618, 23441, 2232, 237, 726, 151, 4221, 5071, 265, 400, 10~
## $ B01001_010M <dbl> 425, 1079, 276, 93, 148, 54, 338, 580, 198, 168, 137, 84, ~
## $ B01001_011E <dbl> 3061, 45914, 3968, 836, 1417, 236, 6902, 9198, 588, 618, 2~
## $ B01001_011M <dbl> 112, 100, 72, 230, 68, 40, 97, 98, 174, 209, 194, 72, 119,~
```

```
## $ B01001_012E <dbl> 2516, 42056, 2580, 515, 1100, 301, 6441, 7228, 839, 489, 2~
## $ B01001_012M <dbl> 90, 100, 50, 115, 90, 80, 79, 121, 243, 86, 179, 101, 22, ~
## $ B01001_013E <dbl> 2098, 38382, 1812, 759, 997, 248, 5997, 5812, 443, 329, 24~
## $ B01001_013M <dbl> 298, 1424, 200, 173, 230, 79, 543, 580, 121, 122, 323, 93,~
## $ B01001_014E <dbl> 1889, 33557, 1581, 377, 1421, 265, 6545, 5461, 503, 197, 2~
## $ B01001_014M <dbl> 282, 1436, 191, 131, 240, 78, 534, 576, 128, 134, 369, 90,~
## $ B01001_015E <dbl> 1838, 36681, 1319, 519, 1344, 337, 6856, 5111, 698, 263, 2~
## $ B01001_015M <dbl> 30, 27, 71, 106, 111, 67, 68, 148, 129, 85, 197, 63, 30, 9~
## $ B01001_016E <dbl> 1857, 34415, 1242, 734, 1527, 374, 6699, 5315, 626, 445, 2~
## $ B01001_016M <dbl> 32, 23, 52, 160, 124, 68, 89, 79, 114, 127, 192, 10, 68, 9~
## $ B01001_017E <dbl> 1510, 30266, 1318, 613, 1787, 322, 6023, 5020, 382, 433, 1~
## $ B01001_017M <dbl> 269, 1083, 169, 115, 143, 76, 424, 412, 102, 85, 292, 70, ~
## $ B01001_018E <dbl> 937, 11276, 634, 331, 275, 112, 2344, 2932, 227, 90, 899, ~
## $ B01001_018M <dbl> 235, 915, 141, 109, 109, 44, 356, 401, 90, 63, 250, 45, 74~
## $ B01001_019E <dbl> 1145, 14990, 684, 406, 495, 144, 3332, 3750, 222, 112, 127~
## $ B01001_019M <dbl> 215, 935, 166, 113, 151, 73, 391, 422, 83, 71, 274, 54, 11~
## $ B01001_020E <dbl> 529, 8277, 398, 154, 369, 114, 1651, 1678, 167, 103, 532, ~
## $ B01001_020M <dbl> 168, 882, 112, 79, 119, 39, 379, 267, 107, 74, 118, 46, 66~
## $ B01001_021E <dbl> 863, 11587, 524, 378, 670, 126, 2500, 2322, 118, 138, 1106~
## $ B01001_021M <dbl> 204, 927, 165, 109, 130, 48, 305, 367, 64, 69, 200, 47, 93~
## $ B01001_022E <dbl> 1178, 13418, 762, 541, 648, 222, 3940, 3519, 314, 344, 127~
## $ B01001_022M <dbl> 220, 751, 160, 115, 149, 53, 356, 401, 72, 96, 205, 58, 13~
## $ B01001_023E <dbl> 653, 8630, 448, 374, 533, 83, 2868, 2080, 185, 105, 949, 1~
## $ B01001_023M <dbl> 188, 589, 88, 68, 102, 37, 381, 323, 61, 47, 139, 48, 84, ~
## $ B01001_024E <dbl> 404, 4893, 200, 57, 275, 55, 1453, 1137, 134, 68, 469, 83,~
## $ B01001_024M <dbl> 112, 540, 68, 41, 86, 32, 241, 275, 60, 38, 121, 42, 85, 3~
## $ B01001_025E <dbl> 327, 4753, 134, 54, 63, 69, 1096, 1005, 66, 142, 265, 56, ~
## $ B01001_025M <dbl> 140, 535, 64, 50, 67, 47, 238, 233, 42, 59, 90, 30, 67, 39~
## $ B01001_026E <dbl> 39778, 542562, 30723, 11521, 20226, 4515, 101265, 104330, ~
## $ B01001_026M <dbl> 196, 90, 86, 242, 69, 92, 98, 280, 125, 261, 165, 88, 164,~
## $ B01001_027E <dbl> 2087, 29947, 3140, 833, 1234, 233, 6407, 6605, 317, 203, 2~
## $ B01001_027M <dbl> 107, 61, 40, 201, 41, 26, 80, 200, 66, 113, 118, 30, 36, 5~
## $ B01001_028E <dbl> 1760, 32463, 2338, 590, 1290, 319, 6455, 6219, 318, 124, 2~
## $ B01001_028M <dbl> 301, 1268, 274, 212, 233, 90, 469, 658, 115, 132, 300, 53,~
## $ B01001_029E <dbl> 2635, 30891, 2025, 892, 1466, 203, 7457, 6190, 390, 56, 26~
## $ B01001_029M <dbl> 271, 1258, 262, 206, 234, 91, 483, 642, 88, 62, 300, 60, 6~
## $ B01001_030E <dbl> 1203, 19764, 1114, 513, 871, 203, 4383, 3579, 228, 39, 157~
## $ B01001_030M <dbl> 72, 40, 51, 106, 30, 44, 78, 145, 96, 48, 77, 41, 24, 25, ~
## $ B01001_031E <dbl> 3533, 15204, 561, 246, 467, 162, 2538, 2869, 91, 61, 714, ~
## $ B01001_031M <dbl> 224, 61, 31, 115, 77, 70, 74, 159, 51, 73, 97, 51, 59, 41,~
## $ B01001_032E <dbl> 1740, 8121, 508, 24, 174, 100, 1582, 1974, 59, 0, 295, 30,~
## $ B01001_032M <dbl> 322, 1141, 180, 30, 106, 44, 391, 593, 46, 20, 139, 28, 57~
## $ B01001_033E <dbl> 1210, 6735, 645, 104, 246, 84, 963, 1524, 22, 0, 295, 42, ~
## $ B01001_033M <dbl> 261, 789, 170, 81, 107, 38, 240, 404, 29, 20, 158, 32, 34,~
## $ B01001_034E <dbl> 2627, 22519, 1942, 564, 790, 190, 3981, 3985, 186, 137, 11~
## $ B01001_034M <dbl> 372, 1158, 250, 185, 152, 78, 414, 534, 48, 74, 237, 48, 8~
## $ B01001_035E <dbl> 2757, 47598, 3278, 648, 1277, 226, 6573, 9064, 318, 212, 2~
## $ B01001_035M <dbl> 79, 86, 57, 154, 88, 38, 77, 130, 63, 95, 143, 30, 35, 86,~
## $ B01001_036E <dbl> 2601, 43691, 2425, 640, 1255, 253, 6138, 7709, 301, 68, 24~
## $ B01001_036M <dbl> 123, 39, 38, 132, 54, 64, 76, 26, 91, 64, 93, 50, 236, 80,~
## $ B01001_037E <dbl> 2569, 41716, 1522, 791, 1078, 226, 6276, 6778, 270, 224, 2~
## $ B01001_037M <dbl> 344, 1485, 266, 201, 206, 80, 503, 539, 91, 90, 291, 58, 1~
## $ B01001_038E <dbl> 1843, 36449, 1958, 466, 1395, 323, 6357, 5466, 221, 197, 2~
## $ B01001_038M <dbl> 320, 1469, 255, 166, 190, 79, 510, 533, 73, 88, 310, 58, 9~
```

```
## $ B01001_039E <dbl> 2152, 38677, 1456, 652, 1522, 255, 6840, 5844, 380, 182, 2~
## $ B01001_039M <dbl> 121, 109, 58, 26, 104, 47, 71, 105, 102, 104, 161, 33, 23,~
## $ B01001_040E <dbl> 2038, 35028, 1497, 731, 1417, 279, 6494, 6232, 467, 366, 2~
## $ B01001_040M <dbl> 99, 37, 21, 22, 90, 60, 81, 87, 107, 55, 155, 8, 68, 50, 4~
## $ B01001_041E <dbl> 2238, 32908, 1776, 899, 1292, 293, 6552, 6489, 420, 400, 2~
## $ B01001_041M <dbl> 258, 1311, 224, 166, 203, 58, 444, 625, 101, 106, 258, 53,~
## $ B01001_042E <dbl> 801, 11608, 632, 284, 417, 131, 1961, 2965, 211, 84, 811, ~
## $ B01001_042M <dbl> 199, 947, 141, 101, 145, 53, 292, 618, 89, 54, 210, 39, 94~
## $ B01001_043E <dbl> 1147, 17670, 725, 423, 771, 182, 3675, 4386, 139, 186, 140~
## $ B01001_043M <dbl> 215, 1090, 165, 155, 163, 54, 377, 479, 64, 96, 239, 43, 8~
## $ B01001_044E <dbl> 796, 10307, 333, 416, 486, 100, 2449, 2184, 89, 150, 750, ~
## $ B01001_044M <dbl> 186, 799, 121, 164, 128, 45, 524, 367, 47, 76, 159, 41, 10~
## $ B01001_045E <dbl> 881, 14241, 705, 314, 623, 144, 2983, 3171, 185, 214, 1268~
## $ B01001_045M <dbl> 209, 1021, 139, 122, 159, 64, 340, 435, 76, 85, 212, 41, 1~
## $ B01001_046E <dbl> 1151, 17554, 954, 513, 880, 233, 4122, 4384, 338, 222, 121~
## $ B01001_046M <dbl> 271, 897, 143, 164, 175, 71, 416, 425, 94, 83, 186, 44, 12~
## $ B01001_047E <dbl> 882, 11546, 730, 442, 594, 180, 3114, 2384, 247, 207, 1375~
## $ B01001_047M <dbl> 185, 857, 147, 148, 125, 47, 344, 315, 74, 71, 190, 58, 75~
## $ B01001_048E <dbl> 649, 8283, 208, 187, 402, 149, 1932, 2242, 148, 202, 485, ~
## $ B01001_048M <dbl> 168, 670, 100, 106, 113, 55, 286, 376, 63, 83, 101, 54, 76~
## $ B01001_049E <dbl> 478, 9642, 251, 349, 279, 47, 2033, 2087, 73, 52, 461, 67,~
## $ B01001_049M <dbl> 163, 669, 117, 134, 101, 30, 348, 272, 46, 43, 149, 33, 80~
## $ geometry    <MULTIPOLYGON [°]> MULTIPOLYGON (((-82.02684 3..., MULTIPOLYGON ~
```

That's a lot of data, so we need to create a subset we can work with later. Let's use the GEOID to identify the state and create a dataset with only those counties in Ohio.

```r
# get just Ohio
ohio <- d.sex.by.age %>%
  mutate(., STATEID = stringr::str_sub(GEOID, 1, 2)) %>%
  # Take a second and think through what the above line is doing
  dplyr::filter(STATEID == "39")


# map it to verify
tmap::tm_shape(ohio) + tm_polygons()
```

Next, we'll formalize our space by creating neighbors, and thus, **W**

- First we'll project
- Next, we'll use Queen contiguity to define **W**

```
# Check it first
sf::st_crs(ohio)
```

```
## Coordinate Reference System:
##   User input: NAD83
##   wkt:
## GEOGCRS["NAD83",
##     DATUM["North American Datum 1983",
##         ELLIPSOID["GRS 1980",6378137,298.257222101,
##             LENGTHUNIT["metre",1]]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["latitude",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["longitude",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4269]]
```

```r
# then reproject to north american equidistant conic
ohio.projected <- ohio %>% sf::st_transform(., "ESRI:102010")

# plot it again to make sure nothing broke
tmap::tm_shape(ohio.projected) + tm_polygons()
```

```r
# make the neighborhood
nb <- spdep::poly2nb(ohio.projected, queen = TRUE)

nb
```

```
## Neighbour list object:
## Number of regions: 88
## Number of nonzero links: 460
## Percentage nonzero weights: 5.940083
## Average number of links: 5.227273
```

For each polygon in our polygon object, `nb` lists all neighboring polygons. For example, to see the neighbors for the first polygon in the object, type:

```r
nb[[1]]
```

```
## [1]  2 66 72
```

Polygon 1 has three neighbors. The numbers represent the polygon IDs as stored in the spatial object `ohio.projected`. Polygon 1 is associated with the `NAME` attribute name `"Erie County"` and its neighboring polygons are associated with the counties:

```
ohio.projected$NAME[1] # county in index 1
```

```
## [1] "Erie County, Ohio"
```

```
nb[[1]] %>% ohio.projected$NAME[.] # and it's neighbors.
```

```
## [1] "Lorain County, Ohio"   "Sandusky County, Ohio" "Huron County, Ohio"
```

```
# Note we're doing this programmatically step-by-step
```

Next, we need to assign weights to each neighboring polygon. In our case, each neighboring polygon will be assigned equal weight `(style="W")`. This is accomplished by assigning the fraction: `1 / ( # of neighbors)` to each neighboring county then summing the weighted values. While this is the most intuitive way to summaries the neighbors' values it has one drawback in that polygons along the edges of the study area will base their lagged values on fewer polygons thus potentially over- or under-estimating the true nature of the spatial autocorrelation in the data. For this example, we'll stick with the `style="W"` option for simplicity's sake but note that other more robust options are available, notably `style="B"`.

```
lw <- spdep::nb2listw(nb, style="W", zero.policy=TRUE)
```

The `zero.policy=TRUE` option allows for lists of non-neighbors. This should be used with caution since the user may not be aware of missing neighbors in their dataset. However, a zero.policy of `FALSE` would return an error if you have a dataset where a polygon does not have a neighbor.

To see the weight of the first polygon's four neighbors type:

```
lw$weights[1]
```

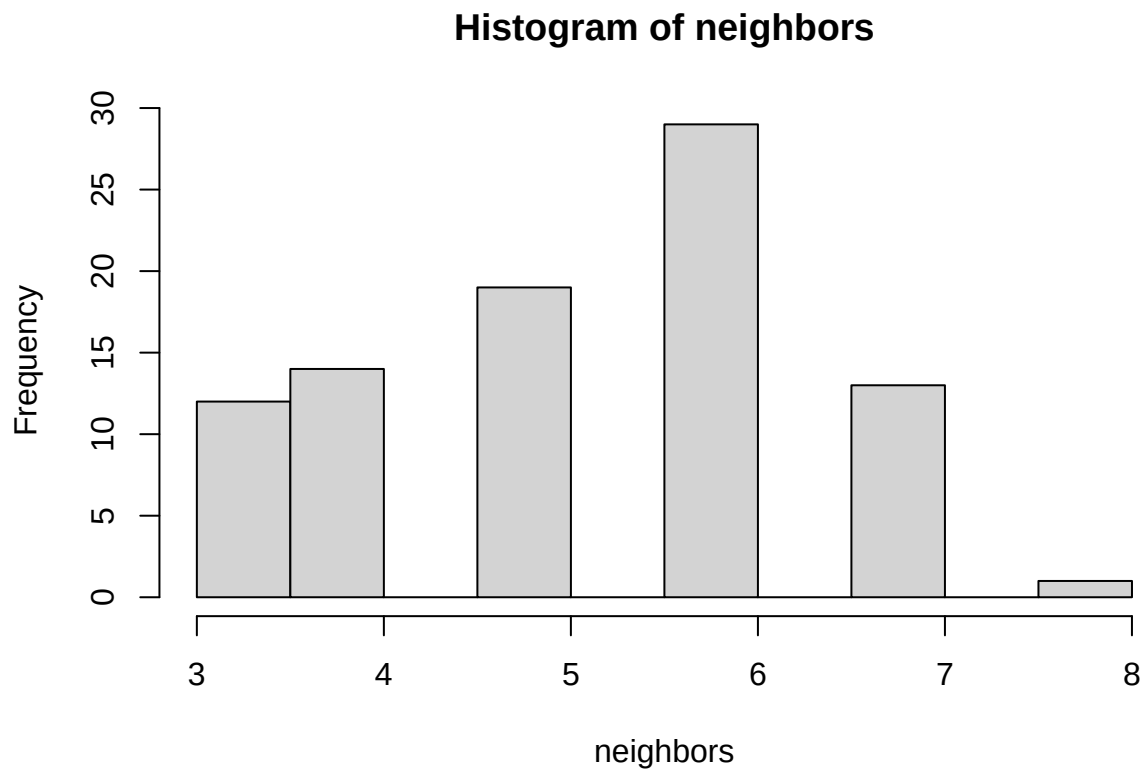
```
## [[1]]
## [1] 0.3333333 0.3333333 0.3333333
```

This row-normalized our weights!

We can also plot the distribution of neighbors across the dataset.

```
# use attr to get the count of neighbors in W
neighbors <- attr(lw$weights,"comp")$d

# plot it
hist(neighbors)
```



Finally, we'll compute the average neighbor population of Females 75-79 years of age for each polygon. These values are often referred to as spatially lagged values. The following table shows the average neighboring F 75-79 values (stored in the F75.lag object) for each county. Note, I determined the correct attribute (variable 47 from table B01001) by reading the metadata from the following link: https://www.census.gov/programs-surveys/acs/technical-documentation/table-shells.2020.html#list-tab-79594641 .

```
F75.lag <- lag.listw(lw, ohio.projected$B01001_047E)
F75.lag
```

```
##  [1] 2314.0000 5989.0000 3753.0000 3480.3333  442.7500 3235.6667  917.0000
##  [8] 4302.5000 2263.1667 1022.7500 1684.6667  550.0000 1836.8571 1060.1429
## [15] 4372.0000 1667.8333  912.4000  936.1667 1296.6667 1397.6667 3469.2500
## [22] 2850.1250 7564.5000 4114.4000 2893.2000 8028.2000  941.0000  976.7500
## [29]  970.0000 1186.2000 1838.6667  574.7500 1592.0000 1264.3333 9055.0000
## [36] 2547.6000 3428.8333  864.7500  835.0000 8303.3333 1390.6667  599.7500
## [43]  612.6667  732.8000  929.0000 1100.6667 3039.2857  783.1667 1013.5000
## [50]  945.4000  978.7143  695.0000 3981.2500 6667.0000 5727.1667 1061.0000
## [57]  944.0000 2679.6667  699.5000 3368.3333  987.8000 1271.2500 2881.6667
## [64] 2476.0000 3893.2000 1233.4000 1084.5000  748.8000 3066.0000 3279.0000
## [71] 1492.7143 1798.1429 1710.2857 3305.2857 6629.7500  879.7500 3231.7143
## [78] 2779.5000  585.4286  656.6000 5395.6667  905.3333  840.6667  683.0000
## [85]  771.1667 2057.5714  938.5000  903.5000
```

Computing Moran's I

To get the Moran's I value, simply use the moran.test function.

```r
moran.test(ohio.projected$B01001_047E, lw)
```

```
##
##  Moran I test under randomisation
##
## data:  ohio.projected$B01001_047E
## weights: lw
##
## Moran I statistic standard deviate = 3.7197, p-value = 9.971e-05
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.204466841      -0.011494253       0.003370742
```

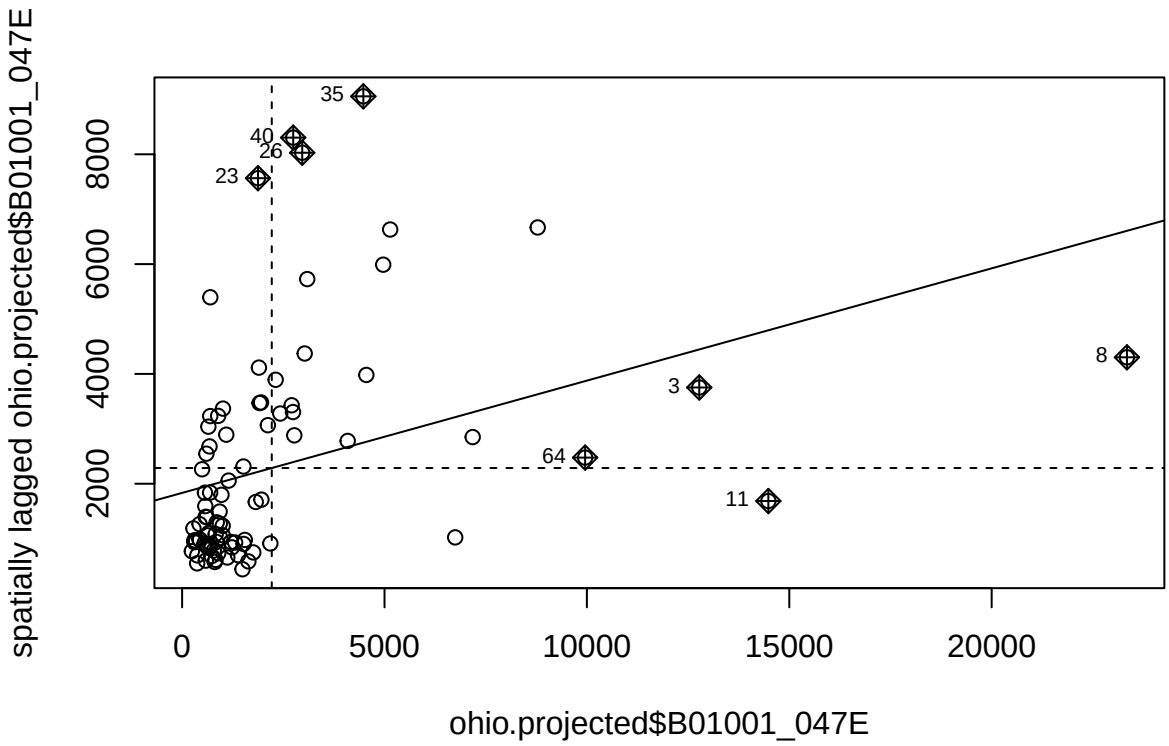
Note that the p-value computed from the `moran.test` function is not computed from an MC simulation but **analytically** instead. This may not always prove to be the most accurate measure of significance. To test for significance using the MC simulation method instead, use the moran.mc function.

Moran's plots

Thus far, our analysis has been a global investigation of spatial autocorrelation. We can also use local indicators of spatial autocorrelation (LISA) to analyze our dataset. One way of doing so is through the use of a Moran plot.

The process to make a plot is relatively simple:

```r
# use zero.policy = T because some polygons don't have neighbors
moran.plot(ohio.projected$B01001_047E, lw, zero.policy=TRUE, plot=TRUE)
```



Your tasks

1. Create a spatial subset of the US, with at AT MINIMUM 4 states, MAXIMUM 7 states. States must be contiguous.
2. Choose a variable. If it's a raw count, you should normalize the variable in an appropriate manner (e.g., by total population, percent, by area)
3. Make a histogram of your chosen variable
4. Make a choropleth map of your chosen variable. Choose an appropriate data classification scheme
5. Develop a contiguity-based spatial weights matrix of your choosing (i.e., rook or queen)
   5.1. Row-standardize the W
   5.2. Plot a histogram of the number of neighbors
   5.3. Calculate the average number of neighbors
   5.4. Make a Moran Plot
6. Repeat #5 (and 5.1 - 5.4) above with a W developed using the IDW method. You will need to investigate the `spdep` documentation to find the correct method/function.

Questions:

1. Describe in your own words how Moran's I is calculated

2. Describe in your own words: what is a spatially-lagged variable?
3. How does your analysis in this lab (as simple as it is) differ by how you have formalized W (e.g., space, neighbors) in two different methods? How might it affect analysis?
4. What does it mean if an observation falls in the "H-L" quadrant? Why might it be useful to detect such occurances?

Bonus (+30 points)

B1. make another Moran plot, this time do so manually (use `geom_point` from `ggplot`). You must label each quadrant with HH, HL, LL, and LH, respectively. You should also use color and/or shape to denote whether an observation is statistically significant. Tip, you can find the data you want using the `moran.plot` function, but you'll have to alter the function call and read some documentation.

B2. plot a choropleth map of your dataset with a categorical color scheme, where the shading corresponds to the Moran plot (really, "LISA") quadrants. Thus, your map will have four shades of color.